Reinforcement Learning for Text Flappy Bird: Monte Carlo Control vs. Sarsa(λ)

Xavier Plantier

CentraleSupélec, Université Paris-Saclay
xavier.plantier@student-cs.fr

Abstract. We study two tabular reinforcement learning agents on the Text Flappy Bird (TFB) environment: an on-policy first-visit Monte Carlo (MC) control agent and a Sarsa(λ) agent with accumulating eligibility traces. Both agents are trained on the `TextFlappyBird-v0` version, which exposes the player's distance to the next pipe gap. We compare their learning dynamics, parameter sensitivity, and ability to generalize to unseen level configurations. Sarsa(λ) consistently learns faster and reaches higher returns, while MC remains competitive once converged. Code is available at `https://github.com/xavierplt/Text-Flappy-Bird--TFB-`.

Keywords: Reinforcement Learning · Monte Carlo · Sarsa(λ) · Eligibility Traces.

1 Introduction

Text Flappy Bird (TFB) is a simplified, text-rendered variant of the Flappy Bird game. The agent controls a single character that must navigate through gaps between vertical pipes by choosing, at each time step, between two actions: *idle* or *flap*. We implement and compare two classical tabular RL agents: Monte Carlo control and Sarsa(λ).

2 Experimental Setup

We use the `TextFlappyBird-v0` environment [2] with default configuration (height $= 15, \text{width} = 20, \text{pipe_gap} = 4$). The observation is a discrete tuple (dx, dy) giving the horizontal and vertical distance from the player to the center of the next pipe gap. The reward is $+1$ per surviving step. Both agents use a tabular Q-table indexed by (dx, dy) and an ε-greedy behavior policy with $\varepsilon_0 = 0.1$ and decay 0.9999 per episode (floor 0.01). We train each agent for 20,000 episodes with discount factor $\gamma = 1$.

Monte Carlo agent. On-policy first-visit MC control with incremental mean updates of $Q(s, a)$.

Sarsa(λ) agent. Tabular Sarsa(λ) with accumulating traces, step-size $\alpha = 0.1$ and trace decay $\lambda = 0.9$:

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t), \quad Q \leftarrow Q + \alpha \delta_t e, \quad e \leftarrow \gamma \lambda e. \tag{1}$$

3 Results and Discussion

3.1 Comparison of the two agents

Figure 1 shows the learning curves of both agents. Sarsa(λ) reaches a high-return regime within a few thousand episodes thanks to its online updates and eligibility traces, which propagate credit immediately. MC, by contrast, only updates at episode end and converges more slowly but eventually attains a comparable greedy-policy performance. Table 1 reports the final metrics. The state-value heatmaps in Fig. 2 show that both agents recover the same qualitative structure: states near the gap center have the highest value.

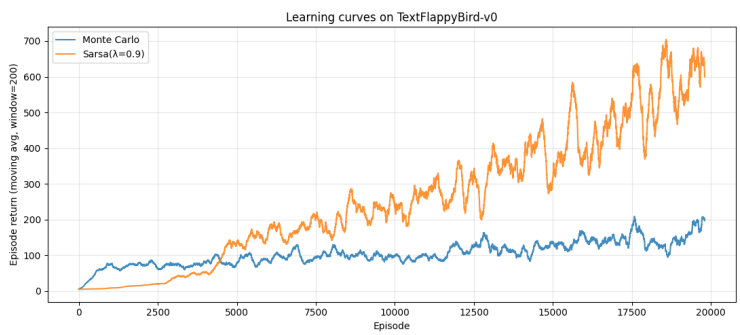
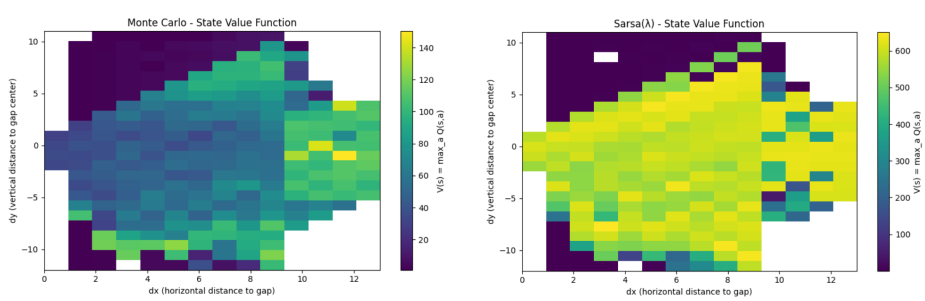


Fig. 1: Learning curves of MC and Sarsa(λ) on TFB (moving average, window 200).



(a) Monte Carlo (b) Sarsa(λ)

Fig. 2: State-value functions $V(s) = \max_a Q(s, a)$ over the (dx, dy) grid.

Table 1: Performance summary on `TextFlappyBird-v0` (h=15, w=20, gap=4). Greedy returns are evaluated over 100 episodes after 20,000 training episodes.

Agent	States learned	Final return (avg)	Greedy max
Monte Carlo	272	186.6	2403
Sarsa($\lambda = 0.9$)	271	643.1	5503

Parameter sensitivity. A sweep over $\lambda \in \{0, 0.3, 0.6, 0.9, 0.99\}$ (Fig. 3) shows that intermediate to high λ values ($\sim$ 0.8–0.9) work best. $\lambda = 0$ (Sarsa(0)) learns much more slowly, while $\lambda = 0.99$ becomes unstable. A sweep over α confirms that $\alpha = 0.1$ offers a good trade-off. MC is fairly robust to ε but very sensitive to the number of training episodes due to its high-variance returns.

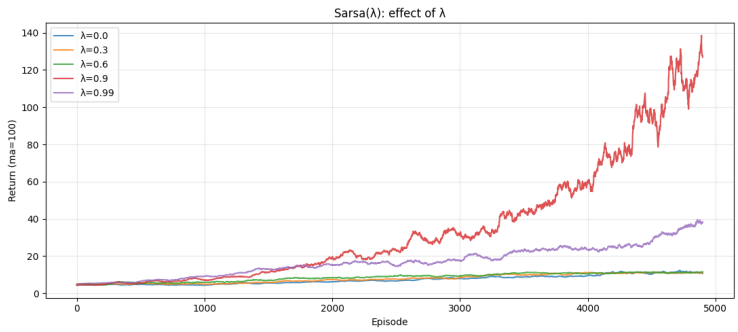


Fig. 3: Sarsa(λ) sensitivity to the trace decay parameter λ.

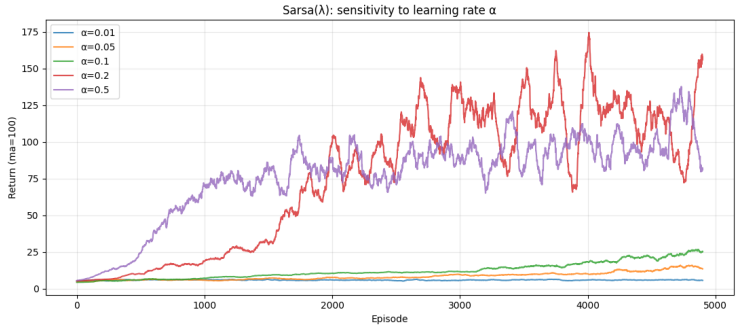


Fig. 4: Sensitivity to the learning rate α.

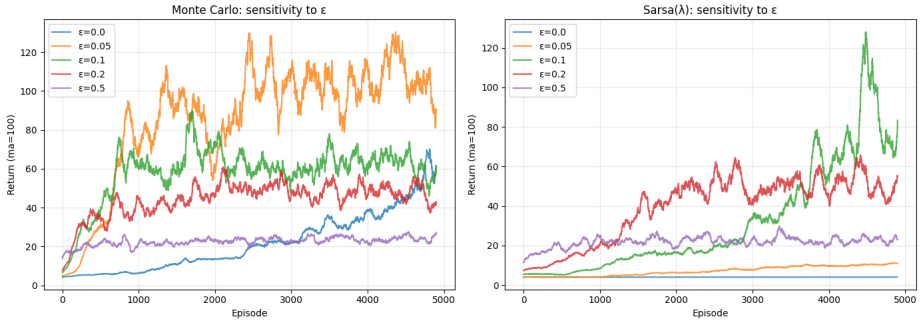


Fig. 5: Sensibility to ϵ.

3.2 The two TFB environments

`TextFlappyBird-v0` returns the compact (dx, dy) tuple, yielding a state space of roughly $w \times h \approx 300$ discrete states, ideal for tabular methods. `TextFlappyBird-screen-v0` returns the full text grid, i.e. a binary matrix of size $h \times w$. The latter has an exponentially larger state space (2^{hw}), which makes naive tabular learning infeasible and effectively requires function approximation. The trade-off is informativeness vs. tractability: the screen version is closer to the original perceptual problem but much harder to learn from scratch; the (dx, dy) version provides a privileged, hand-crafted representation that drastically simplifies the task.

3.3 Transfer to the original Flappy Bird

The original Flappy Bird Gym [3] returns continuous floating-point distances. The two implemented agents *cannot* be used directly because their tabular Q-tables index states by exact tuples and cannot generalize to unseen continuous values. Two adaptations are possible: discretize the continuous observation into bins and reuse the same agents or replace the tabular Q with function approximation.

3.4 Generalization to other configurations

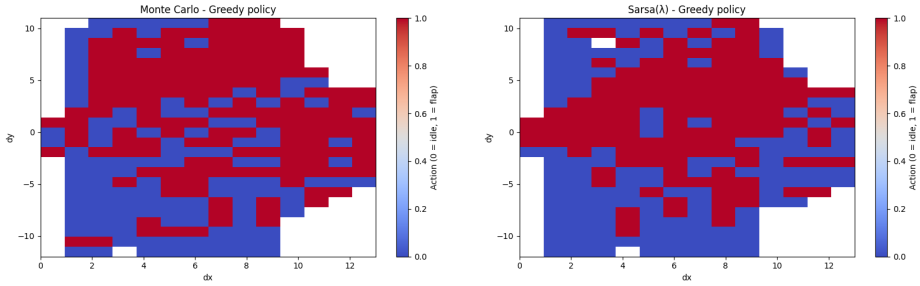
We evaluate the greedy policies of both agents (trained on $h = 15, w = 20, \text{gap} = 4$) on alternative configurations. Results are summarized in Table 2. Performance degrades when the gap shrinks (harder dynamics, unseen states) and improves when the gap widens. Changes in h and w also hurt the agent because the encountered (dx, dy) values fall outside the trained Q-table; for these states the agent falls back to a default action. This shows that tabular agents do not generalize to unseen states: any form of state aggregation or function approximation would be required for true robustness.

Table 2: Transfer of the agents trained on $(15, 20, 4)$ to other configurations (mean return over 30 episodes, greedy policy).

Configuration (h, w, gap)	Monte Carlo	Sarsa(λ)
$(15, 20, 4)$ *(training)*	330	1075
$(15, 20, 3)$	43	14
$(15, 20, 5)$	437	1431
$(15, 20, 6)$	348	1873
$(20, 25, 4)$	4	4
$(10, 15, 4)$	134	443

3.5 Comprehensive results

Across all experiments, Sarsa(λ) dominates MC in terms of sample efficiency and peak greedy return. MC remains a useful baseline because of its conceptual simplicity and unbiased return estimates, but its high variance and episode-end-only updates make it slower. Both agents discover the same essential policy: *flap when below the gap center, idle when above*. This is clearly visible in the greedy-policy heatmaps.



(a) Monte Carlo (b) Sarsa(λ)

Fig. 6: State-value functions $V(s) = \max_a Q(s, a)$ over the (dx, dy) grid.

4 Conclusion

We implemented and compared a Monte Carlo control agent and a Sarsa(λ) agent on the Text Flappy Bird environment. Sarsa(λ) with $\lambda \approx 0.9$ proved to be the most effective combination, learning faster and reaching higher returns. The main limitation of both agents is the lack of generalization, which restricts their use to the exact configuration they were trained on. Function approximation would be the natural next step to address both the screen-based version of the environment and the original Flappy Bird game.

References

1. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. MIT Press, 2nd edn. (2018)
2. Christodoulidis, S.: Text Flappy Bird Gym. `https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym`
3. Tavares, G.: Flappy Bird Gym. `https://github.com/Talendar/flappy-bird-gym`